paiement-api — SEPA transfer microservice — RTS 2024/1774 · Internal policy PDS-2 (Banque Kerlann, fictitious)

Banque Kerlann (fictitious) · paiement-api — SEPA transfer microservice

Prepared by: **Attest (AI-assisted)**

Auditor sign-off: **pending**

Owner: **IT department — Payments development team**

Date: **2026-11-04 08:31**

Run: **20261104-083104-59c3**

1. Executive summary

OPINION AT TESTING

Unsatisfactory

Basis for opinion

9 controls tested (9 conclusive) against: RTS 2024/1774 — Article 16: ICT systems acquisition, development and maintenance · Internal policy PDS-2 (Banque Kerlann, fictitious) — secure development. Tests run on 2026-11-04 08:31, environment: Demonstration pre-production. The opinion rests solely on the verdicts of the deterministic test scripts and on the evidence sealed in the audit trail.

Scope limitation: Work programme and test scripts drafted by AI and not yet reviewed by an auditor; requirements needing document review or interview are not covered; the opinion applies at the test date only.

9 controls tested	**9** findings	**0** findings closed and verified	**9** open findings
0 human approvals	**18** sensitive data items masked in evidence	**5 min 43 s** audit duration	**$0.783 *** measured AI cost



Critical: 1 High: 4 Medium: 4 Low: 0

Key messages

- 9 findings across 9 controls: 1 critical, 4 high, 4 medium.
- 9 findings remain open, with an action plan, an owner and a due date.

2. Scope and method

Engagement	AUD-20261104-59C3
Entity	Banque Kerlann (fictitious)
System	paiement-api — SEPA transfer microservice
System criticality	Supports critical or important functions (DORA)
Environment	Demonstration pre-production
Owner	IT department — Payments development team
Lead auditor	Attest CI

Date	2026-11-04 08:31
Framework	RTS 2024/1774 — Article 16: ICT systems acquisition, development and maintenance · Internal policy PDS-2 (Banque Kerlann, fictitious) — secure development
Audited target	paiement-api code repository (SEPA transfer microservice)
Work programme	existing catalogue reused · runs/studio/20261104-083104-59c3/catalogue.json

In scope

- Source code, configuration and CI pipeline of the repository
- Versioned test datasets

Out of scope

- Dynamic testing (DAST) and penetration testing
- Runtime and production environments
- Git history

Method

1. Controls are derived from the framework (DORA, RTS 2024/1774, internal policy); severity is the one set by the policy.
2. Each test is a stored Python script, statically checked before execution (read-only) and run with a read-only service account.
3. The verdict comes from the script, never from the model. A screenshot is reviewed by Muse Spark to corroborate; on disagreement the control is flagged for review.
4. SAM 3.1 delineates the proving rows and the personal data, which is masked before distribution.
5. Every fix goes through a change request approved by the system owner, applied with a dedicated account, logged, then verified by the same test (DORA 9(4)(e)).
6. Every event and piece of evidence is sealed in a hash-chained audit trail (SHA-256).

Limitations

- Tests cover the configuration reported by the system, not the actual strength of authentication factors.
- AI-drafted scripts and catalogue must be reviewed by an auditor before being considered validated.
- Fixes are drafted by the audit tool itself (self-review threat): closing a finding requires independent approval by the system owner.
- Fictitious entity and data: demonstration.

3. Findings overview

REF.	RULE	TITLE	SEVERITY	TESTED	AFTER	STATUS	DUE DATE
F-01	PDS-2.1	No hardcoded secrets in code or versioned configuration	Critical	Non-compliant		Open	2026-12-04
F-02	PDS-2.2	TLS certificate verification never disabled	High	Non-compliant		Open	2027-02-02
F-03	PDS-2.3	Password hashing uses approved algorithm	High	Non-compliant		Open	2027-02-02
F-04	PDS-2.3	MD5 and SHA-1 forbidden for security use	High	Non-compliant		Open	2027-02-02
F-05	PDS-2.6	Test data declared synthetic with no production export	High	Non-compliant		Open	2027-02-02
F-06	PDS-2.4	CI runs static security analysis on every change	Medium	Non-compliant		Open	2027-05-03
F-07	PDS-2.5	Dependencies pinned to exact version	Medium	Non-compliant		Open	2027-05-03

F-08	PDS-2.5	Dependencies audited in CI	Medium	Non-compliant	Open	2027-05-03
F-09	PDS-2.7	Authentication successes and failures are logged	Medium	Non-compliant	Open	2027-05-03

4. Detailed findings and controls

F-01 PDS-2.1 No hardcoded secrets in code or versioned configuration

Critical · Non-compliant · Open

CRITERIA

Ensure passwords, keys and tokens are not in clear text and come from vault or environment variables.

PDS-2.1 · RTS 7(2)

Threshold source: PDS-2.1

> *« Aucun secret (mot de passe, clé, jeton) n'est écrit en clair dans le code source ni dans la configuration versionnée : les secrets viennent d'un coffre ou de variables d'environnement. »*

CONDITION

Observed: `{"hardcoded_search_matches": [{"line": 6, "path": "app/config.py", "text": "DB_PASSWORD = \"Az••••\" # TODO : déplacer dans le coffre-fort"}, {"line": 7, "path…`

Expected: `Zero matches with clear-text secret literals; all secrets reference os.environ, os.getenv or vault client.`

Checked 11 files with 2 search hits and 2 local hits and 1 env or vault refs seen with config verified and no instruction-like text found.

PROBABLE CAUSE

Developer convenience hardcoding credentials for local testing. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Hardcoded credentials exposure leading to unauthorized access to production and data.

RECOMMENDATION

Remove hardcoded secrets and load them from a vault or environment variables.

Due date: 2026-12-04 · Owner: IT department — Payments development team

Test procedure: List files via GET /api/repo/paiement-api/tree, search via GET /api/repo/paiement-api/search?q=(?i)(password|passwd|pwd|secret|api[_-]?key|token)\s*=\s*['\"][^'\"]+['\"] and via GET /api/repo/paiement-api/search?q=os\.environ|os\.getenv|vault|secrets\.manager, then read app/config.py and any config files via GET /api/repo/paiement-api/file?path=<file> to verify secret sourcing.

Banque Kerlann · dépôts de code paiement-api

app/config.py

```
1  """Configuration du service."""
2  import os
3
4  PSP_URL = os.getenv("PSP_URL", "https://psp.kerlann-banque.example/v1/transfers")
5  DB_URL = "postgresql://[redacted]:5432/paiement"
6  DB_PASSWORD = '[redacted]'  # TODO : déplacer dans le coffre-fort
7  JWT_SECRET = '[redacted]'
8  TIMEOUT_S = 10
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. Screenshot shows DB_PASSWORD and JWT_SECRET assigned as literals without os.getenv with TODO to move to vault, matching the reported failure. · 3/3 sensitive data items masked (0 spotted by the AI, the rest by the deterministic safety net)
`2026-11-04 08:33:10 · SHA-256 f1d137d27ca60583…`

F-02 PDS-2.2 TLS certificate verification never disabled

High | Non-compliant | Open

CRITERIA

Ensure outgoing calls verify server TLS certificates.

PDS-2.2 · DORA 9(3)(a), RTS 6(2)(a)

Threshold source: PDS-2.2

> *« Tout appel réseau sortant vérifie le certificat TLS du serveur : la vérification n'est jamais désactivée (`verify=False` interdit). »*

CONDITION

Observed: `{"total_hits": 1, "matches": [{"line": 18, "path": "app/payments.py", "text": "response = requests.post(PSP_URL, json=payload, timeout=TIMEOUT_S, verify=False)"…`

Expected: `Zero matches for verify=False, CERT_NONE, unverified context or disabled hostname check.`

Found 1 TLS verification bypass pattern(s) in repository code.

PROBABLE CAUSE

Temporary bypass of TLS verification left in code for local development. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Man-in-the-middle interception of SEPA transfer data and credentials.

RECOMMENDATION

Remove any TLS verification bypass and retain default certificate verification.

Due date: 2027-02-02 · Owner: IT department — Payments development team

Test procedure: Search via GET /api/repo/paiement-api/search?q=verify\s*=\s*False and via GET /api/repo/paiement-api/search?q=CERT_NONE|_create_unverified_context|check_hostname\s*=\s*False, list files via GET /api/repo/paiement-api/tree and read network client files via GET /api/repo/paiement-api/file?path=<file>.

Banque Kerlann · dépôts de code paiement-api

app/payments.py

```
1  """Initiation de virements SEPA via le prestataire de paiement (PSP)."""
2  import logging
3
4  import requests
5
6  from .config import PSP_URL, TIMEOUT_S
7
8  log = logging.getLogger("paiement")
9
10
11 def initiate_transfer(debtor_iban: str, creditor_iban: str, amount_cents: int, reference: str) -> dict:
12     payload = {
13         "debtor": {"iban": debtor_iban},
14         "creditor": {"iban": creditor_iban},
15         "amount": {"value": amount_cents, "currency": "EUR"},
16         "reference": reference,
17     }
18     response = requests.post(PSP_URL, json=payload, timeout=TIMEOUT_S, verify=False)
19     response.raise_for_status()
20     log.info("virement initié ref=%s", reference)
21     return response.json()
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. Screenshot shows line 18 using verify=False in the PSP request.
`2026-11-04 08:32:13 · SHA-256 03ffbdd593d56b5c…`

F-03 PDS-2.3 Password hashing uses approved algorithm

High · Non-compliant · Open

CRITERIA

Ensure passwords are hashed with argon2, bcrypt, scrypt or PBKDF2.

PDS-2.3 · RTS 6(3) et 6(4)

Threshold source: PDS-2.3

> *« Les mots de passe sont hachés avec argon2, bcrypt, scrypt ou PBKDF2. MD5 et SHA-1 sont interdits pour tout usage de sécurité. »*

CONDITION

Observed: `{"path": "app/auth.py", "search_matches": 0, "snippet": "\"\"\"Authentification des opérateurs back-office.\"\"\"\nimport hashlib\n\nfrom .db import get_operato…`

Expected: `Password hashing code uses one of argon2, bcrypt, scrypt or PBKDF2.`

No approved password hashing algorithm was found in password handling files.

PROBABLE CAUSE

Use of legacy example code with fast hashes. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Weak password storage enabling credential recovery after database disclosure.

RECOMMENDATION

Migrate password hashing to argon2, bcrypt, scrypt or PBKDF2.

Due date: 2027-02-02 · Owner: IT department — Payments development team

Test procedure: Search via GET /api/repo/paiement-api/search?q=argon2|bcrypt|scrypt|pbkdf2|passlib and read password handling files via GET /api/repo/paiement-api/file?path=<file> to confirm hashing call for passwords.

Banque Kerlann · dépôts de code paiement-api

app/auth.py

```
1  """Authentification des opérateurs back-office."""
2  import hashlib
3
4  from .db import get_operator
5
6
7  def hash_password(password: str) -> str:
8      return hashlib.md5(password.encode()).hexdigest()
9
10
11 def login(username: str, password: str) -> bool:
12     operator = get_operator(username)
13     if operator is None:
14         return False
15     return operator["password_hash"] == hash_password(password)
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. The screenshot shows app/auth.py hashing passwords with MD5 instead of an approved algorithm.

2026-11-04 08:32:37 · SHA-256 9b670c1eafd915cb…

F-04 PDS-2.3 MD5 and SHA-1 forbidden for security use

High | Non-compliant | Open

CRITERIA

Ensure MD5 and SHA-1 are not used for security purposes.

PDS-2.3 · RTS 6(3) et 6(4)

Threshold source: PDS-2.3

> *« Les mots de passe sont hachés avec argon2, bcrypt, scrypt ou PBKDF2. MD5 et SHA-1 sont interdits pour tout usage de sécurité. »*

CONDITION

Observed: `{"total_search_matches": 1, "security_violations": ["app/auth.py:8:return hashlib.md5(password.encode()).hexdigest()"]}`

Expected: `Zero matches for MD5 or SHA-1 in security context.`

Found 1 MD5 or SHA-1 use in security context in app/auth.py.

PROBABLE CAUSE

Reuse of legacy hashing code. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Collision and brute-force attacks due to broken hash functions.

RECOMMENDATION

Replace MD5 and SHA-1 with approved password hashing.

Due date: 2027-02-02 · Owner: IT department — Payments development team

Test procedure: Search via GET /api/repo/paiement-api/search?q=hashlib\.(md5|sha1)|\bMD5\b|\bSHA-?1\b|hashlib\.new\(['\"]?(md5|sha1) and read matches via GET /api/repo/paiement-api/file?path=<file> to exclude non-security use.

Banque Kerlann · dépôts de code paiement-api

app/auth.py

```
1  """Authentification des opérateurs back-office."""
2  import hashlib
3
4  from .db import get_operator
5
6
7  def hash_password(password: str) -> str:
8      return hashlib.md5(password.encode()).hexdigest()
9
10
11 def login(username: str, password: str) -> bool:
12     operator = get_operator(username)
13     if operator is None:
14         return False
15     return operator["password_hash"] == hash_password(password)
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. Screenshot shows line 8 using hashlib.md5 for password hashing in app/auth.py.
`2026-11-04 08:33:56 · SHA-256 9b670c1eafd915cb…`

F-05 PDS-2.6 Test data declared synthetic with no production export

High | Non-compliant | Open

CRITERIA

Ensure test data is synthetic with synthetic true and no production dump versioned.

PDS-2.6 · RTS 16(5)(a)

Threshold source: PDS-2.6

> *« Les données de test sont synthétiques et déclarées comme telles (champ `"synthetic": true`) ; aucun export de production n'est versionné. »*

CONDITION

Observed: `{"fixtures": ["tests/fixtures/clients_export_prod.json", "tests/fixtures/operateurs_synthetiques.json"], "with_synthetic_true": ["tests/fixtures/operateurs_synt…`

Expected: `Fixture files contain "synthetic": true and tree shows no production export or dump files.`

Checked 2 fixture file(s): 1 with synthetic true, 1 missing it, and 1 production export or dump file(s) in tree.

PROBABLE CAUSE

Copy of production data for realistic testing. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Production personal and payment data exposure in non-production via versioned exports.

RECOMMENDATION

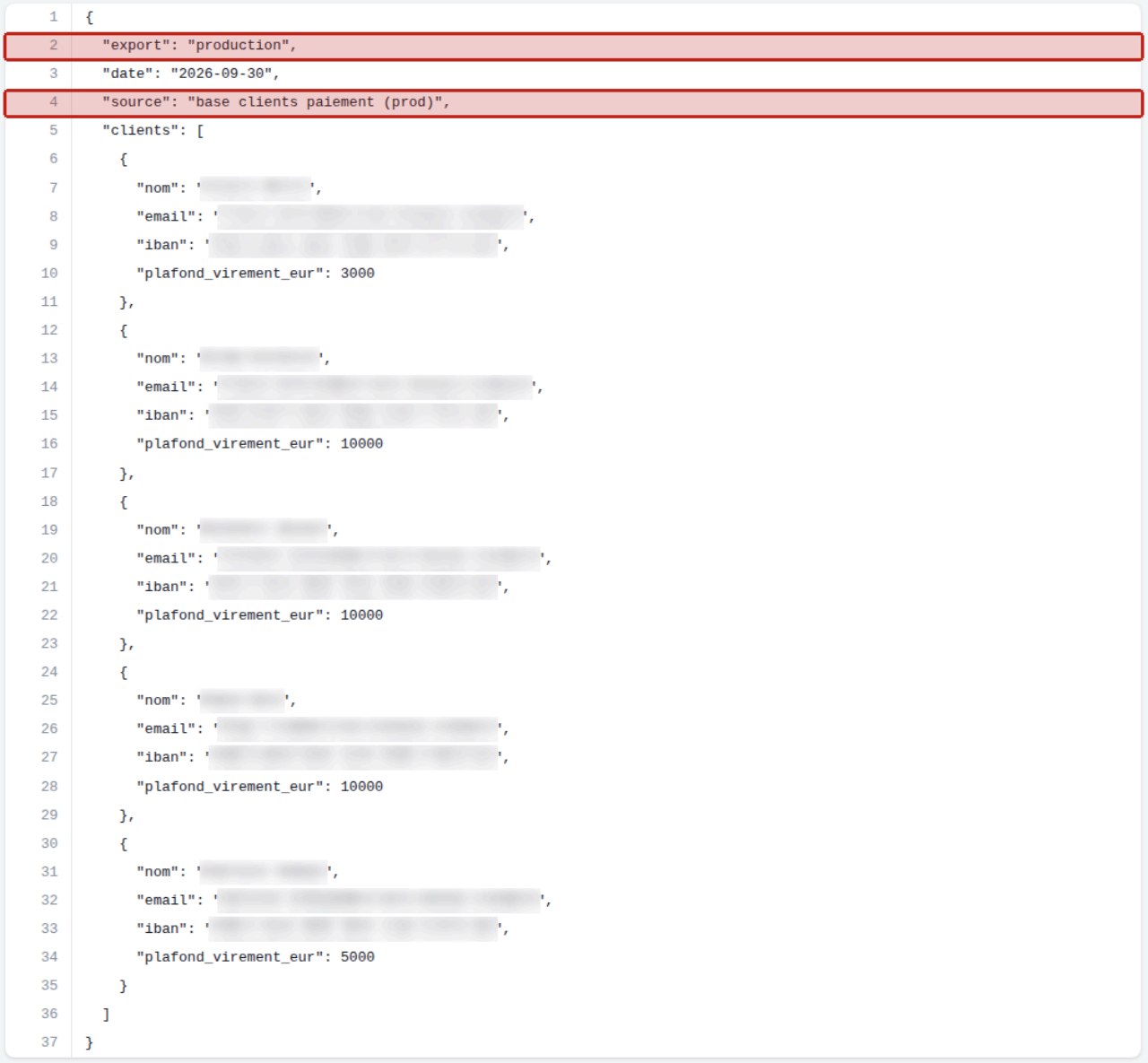
Replace test data with synthetic data flagged synthetic true and remove production exports.

Due date: 2027-02-02 · Owner: IT department — Payments development team

Test procedure: List files via GET /api/repo/paiement-api/tree to review tests and fixtures, search via GET /api/repo/paiement-api/search?q=synthetic and read fixture files via GET /api/repo/paiement-api/file?path=<file> to verify declaration.

Banque Kerlann · dépôts de code paiement-api

tests/fixtures/clients_export_prod.json



```
{
  "export": "production",
  "date": "2026-09-30",
  "source": "base clients paiement (prod)",
  "clients": [
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 3000
    },
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 10000
    },
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 10000
    },
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 10000
    },
    {
      "nom": '[redacted]',
      "email": '[redacted]',
      "iban": '[redacted]',
      "plafond_virement_eur": 5000
    }
  ]
}
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. The file declares a production export with no synthetic flag visible in the content. · 15/15 sensitive data items masked (0 spotted by the AI, the rest by the deterministic safety net)
`2026-11-04 08:35:16 · SHA-256 d603feab519b9c27…`

F-06 PDS-2.4 CI runs static security analysis on every change

Medium · Non-compliant · Open

CRITERIA

Verify CI executes bandit, semgrep or CodeQL on each modification.

PDS-2.4 · RTS 16(3)

Threshold source: PDS-2.4

> *« La chaîne d'intégration continue exécute une analyse statique de sécurité (bandit, semgrep ou CodeQL) à chaque modification. »*

CONDITION

Observed: `{"workflows_checked": [".github/workflows/ci.yml"], "checks": {".github/workflows/ci.yml": {"push": true, "pull_request": false, "scanner": false}}, "search_mat…`

Expected: `A workflow triggered on push and pull_request contains a step invoking bandit, semgrep or CodeQL.`

No workflow triggered on push and pull_request invokes bandit, semgrep or CodeQL.

PROBABLE CAUSE

Security scanning omitted from initial pipeline setup. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Undetected code vulnerabilities reaching production without static review.

RECOMMENDATION

Add bandit, semgrep or CodeQL step to CI on every push and pull request.

Due date: 2027-05-03 · Owner: IT department — Payments development team

Test procedure: List files via GET /api/repo/paiement-api/tree to find .github/workflows files, read each via GET /api/repo/paiement-api/file?path=.github/workflows/ci.yml and search via GET /api/repo/paiement-api/search?q=bandit|semgrep|CodeQL|codeql to confirm analysis step and trigger.



At testing — The screenshot corroborates the test. The workflow triggers only on push and its steps contain no bandit, semgrep or CodeQL invocation.

2026-11-04 08:34:51 · SHA-256 cfbb2e4159bb03c3…

F-07 PDS-2.5 Dependencies pinned to exact version

Medium · Non-compliant · Open

CRITERIA

Ensure every dependency is pinned with == to an exact version.

PDS-2.5 · RTS 16(4) et 16(8)

Threshold source: PDS-2.5

> *« Les dépendances sont épinglées à une version exacte (`paquet==version`) et auditées en intégration continue (pip-audit ou équivalent). »*

CONDITION

Observed: `["flask", "requests>=2.0"]`

Expected: `Every dependency line in requirements.txt uses == exact pin with no >=, ~=, *, or unpinned entries.`

2 of 3 dependency lines are not exact == pins.

PROBABLE CAUSE

Use of flexible version specifiers for convenience. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

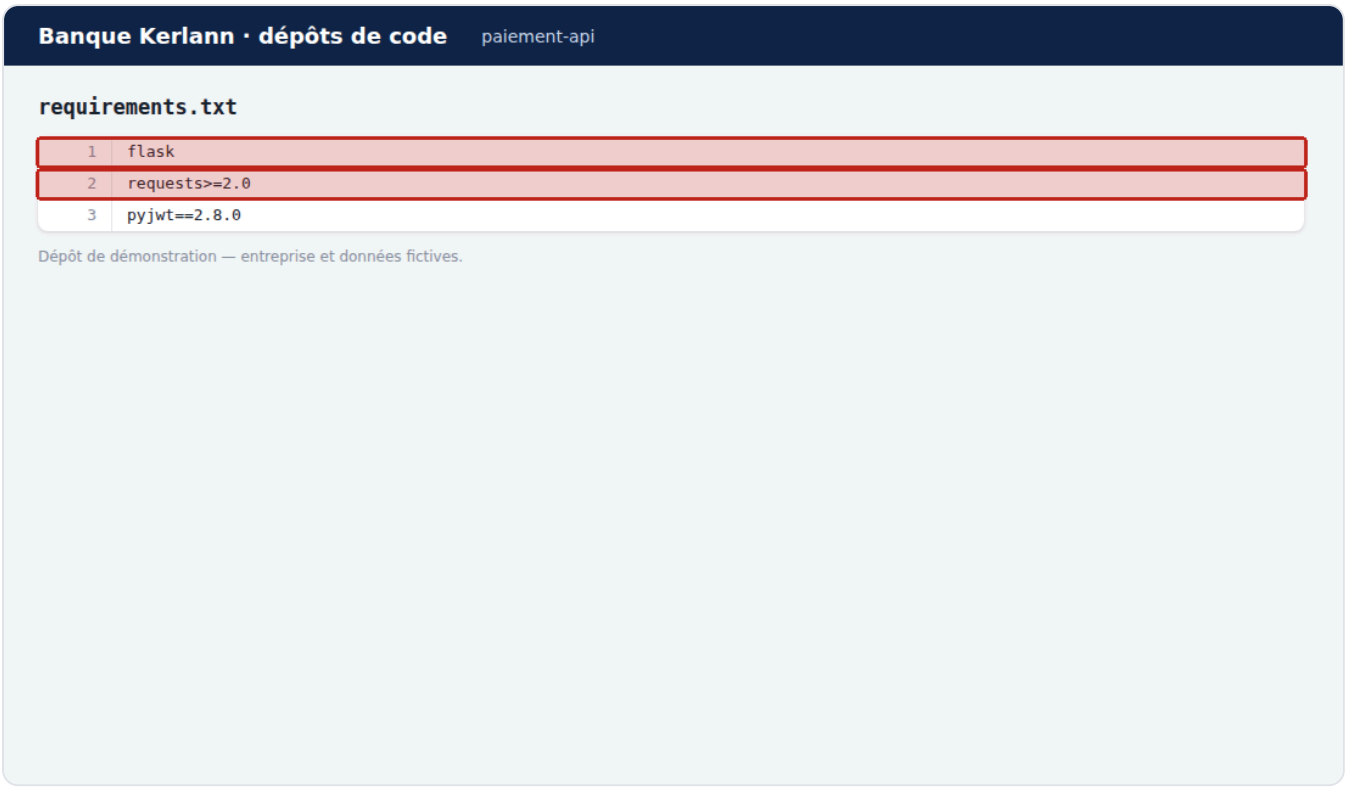
Non-reproducible builds and silent introduction of vulnerable transitive versions.

RECOMMENDATION

Pin all dependencies to exact versions with ==.

Due date: 2027-05-03 · Owner: IT department — Payments development team

Test procedure: List files via GET /api/repo/paiement-api/tree, read requirements.txt via GET /api/repo/paiement-api/file?path=requirements.txt and verify each non-comment line matches exact pin pattern.



At testing — The screenshot corroborates the test. Screenshot shows flask unpinned and requests>=2.0, matching the two non-exact pins reported.

`2026-11-04 08:34:07 · SHA-256 94071ff0cc3ec301…`

F-08 PDS-2.5 Dependencies audited in CI

Medium | Non-compliant | Open

CRITERIA

Ensure dependencies are audited in CI with pip-audit or equivalent.

PDS-2.5 · RTS 16(4) et 16(8)

Threshold source: PDS-2.5

> *« Les dépendances sont épinglées à une version exacte (`paquet==version`) et auditées en intégration continue (pip-audit ou équivalent). »*

CONDITION

Observed: `name: ci on: [push] jobs: test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-python@v5 with: …`

Expected: `CI workflow contains a step invoking pip-audit or equivalent audit on dependencies.`

CI workflow ci.yml contains no pip-audit or equivalent audit step.

PROBABLE CAUSE

Vulnerability scanning not added to pipeline. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Known vulnerable packages deployed without detection at integration.

RECOMMENDATION

Add pip-audit or equivalent audit step to CI.

Due date: 2027-05-03 · Owner: IT department — Payments development team

Test procedure: List files via GET /api/repo/paiement-api/tree, read workflow files via GET /api/repo/paiement-api/file?path=.github/workflows/ci.yml and search via GET /api/repo/paiement-api/search?q=pip-audit|safety|dependabot|osv-scanner to confirm audit step.

Banque Kerlann · dépôts de code paiement-api

.github/workflows/ci.yml

```
1  name: ci
2  on: [push]
3  jobs:
4    test:
5      runs-on: ubuntu-latest
6      steps:
7        - uses: actions/checkout@v4
8        - uses: actions/setup-python@v5
9          with:
10           python-version: "3.12"
11       - run: pip install -r requirements.txt pytest
12       - run: pytest -q
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. The displayed ci.yml shows only checkout, setup-python, install and pytest steps with no audit step.
2026-11-04 08:35:31 · SHA-256 33d3be279bb99b2b…

F-09 PDS-2.7 Authentication successes and failures are logged

Medium | Non-compliant | Open

CRITERIA

Ensure the application logs both successful and failed authentications.

PDS-2.7 · RTS 12(2)(c)(i)

Threshold source: PDS-2.7

> *« Les réussites et les échecs d'authentification sont journalisés par l'application. »*

CONDITION

Observed: `{"success_log": "", "failure_log": "", "success_file": "", "failure_file": "", "files_checked": ["app/auth.py"]}`

Expected: `Code contains logging calls covering both authentication success and authentication failure events.`

Checked repository files and did not find logging for both authentication success and failure.

PROBABLE CAUSE

Logging limited to errors without authentication events. (AI hypothesis, not tested: to be confirmed with the owner)

RISK

Undetected credential attacks and missing audit trail for access review.

RECOMMENDATION

Add application logging for both authentication successes and failures.

Due date: 2027-05-03 · Owner: IT department — Payments development team

Test procedure: Search via GET /api/repo/paiement-api/search?q=log.*auth|auth.*log|login.*log|logger\.(info|warning|error) and via GET /api/repo/paiement-api/search?q=authentication|login_failed|login_success, then read matches via GET /api/repo/paiement-api/file?path=<file>.

Banque Kerlann · dépôts de code paiement-api

app/auth.py

```
"""Authentification des opérateurs back-office."""
import hashlib

from .db import get_operator


def hash_password(password: str) -> str:
    return hashlib.md5(password.encode()).hexdigest()


def login(username: str, password: str) -> bool:
    operator = get_operator(username)
    if operator is None:
        return False
    return operator["password_hash"] == hash_password(password)
```

Dépôt de démonstration — entreprise et données fictives.

At testing — The screenshot corroborates the test. The login function returns False and a password comparison result with no logging calls visible.

2026-11-04 08:36:47 · SHA-256 5b3695ed7ed9f7b6…

Appendices

A. Traceability matrix

REQUIREMENT	RULE	CONTROL	SCRIPT (SHA-256)	RESULT
RTS 7(2)	PDS-2.1	C1 · No hardcoded secrets in code or versioned configuration	7f0048a28697ac6e · written by AI for this engagement	Non-compliant
DORA 9(3)(a), RTS 6(2)(a)	PDS-2.2	C2 · TLS certificate verification never disabled	b27c16657a8d2b96 · written by AI for this engagement	Non-compliant
RTS 6(3) et 6(4)	PDS-2.3	C3 · Password hashing uses approved algorithm	8d6eef47e675ce93 · written by AI for this engagement	Non-compliant
RTS 6(3) et 6(4)	PDS-2.3	C4 · MD5 and SHA-1 forbidden for security use	1f258df4cb2cae36 · written by AI for this engagement	Non-compliant
RTS 16(3)	PDS-2.4	C5 · CI runs static security analysis on every change	718b3effe3b21709 · written by AI for this engagement	Non-compliant
RTS 16(4) et 16(8)	PDS-2.5	C6 · Dependencies pinned to exact version	ca118dc93e49d9df · written by AI for this engagement	Non-compliant
RTS 16(4) et 16(8)	PDS-2.5	C7 · Dependencies audited in CI	354f7a51f59c9db0 · written by AI for this engagement	Non-compliant
RTS 16(5)(a)	PDS-2.6	C8 · Test data declared synthetic with no production export	d08d71cc1d94b3c3 · written by AI for this engagement	Non-compliant
RTS 12(2)(c)(i)	PDS-2.7	C9 · Authentication successes and failures are logged	0dc92f4efae98256 · written by AI for this engagement	Non-compliant

B. Evidence register

FILE			SHA-256
evidence/C1_before.png	C1 · At testing	2026-11-04 08:33:10	f1d137d27ca60583cb9ab6fdeeb5f95fd1610ca96053b4af1fa078191f433317
evidence/C2_before.png	C2 · At testing	2026-11-04 08:32:13	03ffbdd593d56b5c2617e7935507798b8d74d6a1c2264c4c5f0bd6bf9de88b32
evidence/C3_before.png	C3 · At testing	2026-11-04 08:32:37	9b670c1eafd915cb6df201edf42dc67d9cd479c769ad7b5f49bc574e294bd891
evidence/C4_before.png	C4 · At testing	2026-11-04 08:33:56	9b670c1eafd915cb6df201edf42dc67d9cd479c769ad7b5f49bc574e294bd891
evidence/C5_before.png	C5 · At testing	2026-11-04 08:34:51	cfbb2e4159bb03c3141060aef9f46daabbc947f5fddcfe451d46811938fcb0b0
evidence/C6_before.png	C6 · At testing	2026-11-04 08:34:07	94071ff0cc3ec3013b5c4620d989ef9e69def04847069f8c7a03fba625a6574f
evidence/C7_before.png	C7 · At testing	2026-11-04 08:35:31	33d3be279bb99b2bcb2b1d28b179d7bc33bf40aaaa085a8d53a7242c21d9f780

evidence/C8_before.png	C8 · At testing	2026-11-04 08:35:16	d603feab519b9c2747e18ddb985fe13abae028ac459dbb8cdb26de0b430c35b0
evidence/C9_before.png	C9 · At testing	2026-11-04 08:36:47	5b3695ed7ed9f7b6a350dee93a53b6e39a0c10c7fab13e31e1b5531030e26129

C. AI usage, costs and human oversight

MODEL	CALLS	INPUT TOKENS	OUTPUT TOKENS	IMAGES	COST (USD)
muse-spark-1.3	20	27,737	176,191	0	0.7835
sam-3.1	9	0	0	9	price not published
Total					**0.7835**

Sensitive data (personal data and secrets) in evidence: 0/18 spotted by Muse Spark + SAM, 18/18 masked in the end thanks to the deterministic safety net (page text).

* sam-3.1: price not published (pricing.toml)

Safeguards

- Scripts: allow-listed imports, no write requests (static analysis before execution).
- Read-only test account: any write would be refused by the system (defence in depth).
- Fixes: allow-listed route and parameters, human approval, separate change account.
- Segregation of duties: the approver cannot be the auditor.

D. Integrity

Audit-trail fingerprint when the report was issued. Verify with `python -m attest verify <folder>`.

d1b64b97e8a93a0a8ffb52ef106206282de516e2e532155bf41f7f87c9c48ea6

F. Framework coverage

20 requirements read: 12 tested automatically (9 controls; 5 regulation clauses covered through the policy), 8 left to document review, 0 to interviews, 0 rejected for lack of an exact quote. Attest only covers the first group.

REF.	TEXT (VERIFIED QUOTE)	TESTABILITY	CONTROL	
RTS 16(1)	« financial entities shall develop, document and implement a policy governing the acquisition, development, and maintenance of ICT systems. »	document review	0	Policy existence and documentation cannot be verified from source code alone.
RTS 16(1)(a)	« identify security practices and methodologies relating to the acquisition, development, and maintenance of ICT systems; »	document review	0	Policy content requires document review and is not visible in the code repository.
RTS 16(1)(c)	« specify measures to mitigate the risk of unintentional alteration or intentional manipulation of the ICT systems during development, maintenance, and deployment. »	document review	0	Specified measures require policy document review, not code content.
RTS 16(2)	« financial entities shall develop, document, and implement an ICT systems' acquisition, development, and maintenance procedure for the testing and approval of all ICT systems prior to their use and after maintenance. »	document review	0	Procedure existence and approval workflow require document review beyond the repo gateway.

RTS 16(3)	« the procedure shall contain source code reviews covering both static and dynamic testing, including security testing for internet-exposed systems and applications. »	**tested automatically**	→ PDS-2.4	Implemented by PDS-2.4 which is verifiable via CI configuration in the repository.
RTS 16(3)(a)	« identify and analyse vulnerabilities and anomalies in the source code; »	**tested automatically**	→ PDS-2.4	Implemented by PDS-2.4 static analysis verifiable via CI configuration and search.
RTS 16(3)(b)	« adopt an action plan to address those vulnerabilities and anomalies; »	**document review**	0	An action plan is a governance document not observable through the read-only code gateway.
RTS 16(3)(c)	« monitor the implementation of that action plan. »	**document review**	0	Monitoring evidence requires progress tracking documents, not repo file content.
RTS 16(4)	« the procedure shall contain security testing of software packages no later than at the integration phase. »	**tested automatically**	→ PDS-2.5	Implemented by PDS-2.5 dependency audit verifiable via requirements and CI files.
RTS 16(5)(a)	« non-production environments only store anonymised, pseudonymised, or randomised production data; »	**tested automatically**	→ PDS-2.6	Implemented by PDS-2.6 synthetic test data rule verifiable via fixtures and search.
RTS 16(5)(b)	« financial entities are to protect the integrity and confidentiality of data in non-production environments. »	**document review**	0	Environment protections require infrastructure configuration review, not visible in application code.
RTS 16(7)	« the procedure shall include controls to protect the integrity of source code of ICT systems that are developed in-house or by an ICT third-party service provider. »	**document review**	0	Integrity controls such as branch protection and signing are not exposed by the file tree and search gateway.
RTS 16(8)	« proprietary software and, where feasible, the source code provided by ICT third-party service providers, as well as source code coming from open-source projects, are to be analysed and tested in accordance with paragraph »	**tested automatically**	→ PDS-2.4, PDS-2.5	Implemented by PDS-2.4 and PDS-2.5 checks verifiable via CI and dependency files.
PDS-2.1	« Aucun secret (mot de passe, clé, jeton) n'est écrit en clair dans le code source ni dans la configuration versionnée : les secrets viennent d'un coffre ou de variables d'environnement. »	**tested automatically**	1	Hardcoded secret patterns can be searched and read through the read-only repo gateway.
PDS-2.2	« Tout appel réseau sortant vérifie le certificat TLS du serveur : la vérification n'est jamais désactivée (`verify=False` interdit). »	**tested automatically**	1	TLS verification disabling patterns can be searched across the repository.
PDS-2.3	« Les mots de passe sont hachés avec argon2, bcrypt, scrypt ou PBKDF2. MD5 et SHA-1 sont interdits pour tout usage de sécurité. »	**tested automatically**	2	Hashing library imports and uses can be searched and read in source files.
PDS-2.4	« La chaîne d'intégration continue exécute une analyse statique de sécurité (bandit, semgrep ou CodeQL) à chaque modification. »	**tested automatically**	1	Workflow files under .github/workflows can be listed and read via the gateway.

PDS-2.5	« Les dépendances sont épinglées à une version exacte (`paquet==version`) et auditées en intégration continue (pip-audit ou équivalent). »	**tested automatically**	2	Requirements files and workflow files can be listed, read and searched via the gateway.
PDS-2.6	« Les données de test sont synthétiques et déclarées comme telles (champ `"synthetic": true`) ; aucun export de production n'est versionné. »	**tested automatically**	1	Test fixtures can be listed, searched for the synthetic flag and read via the gateway.
PDS-2.7	« Les réussites et les échecs d'authentification sont journalisés par l'application. »	**tested automatically**	1	Logging statements for authentication can be searched and read in application source.